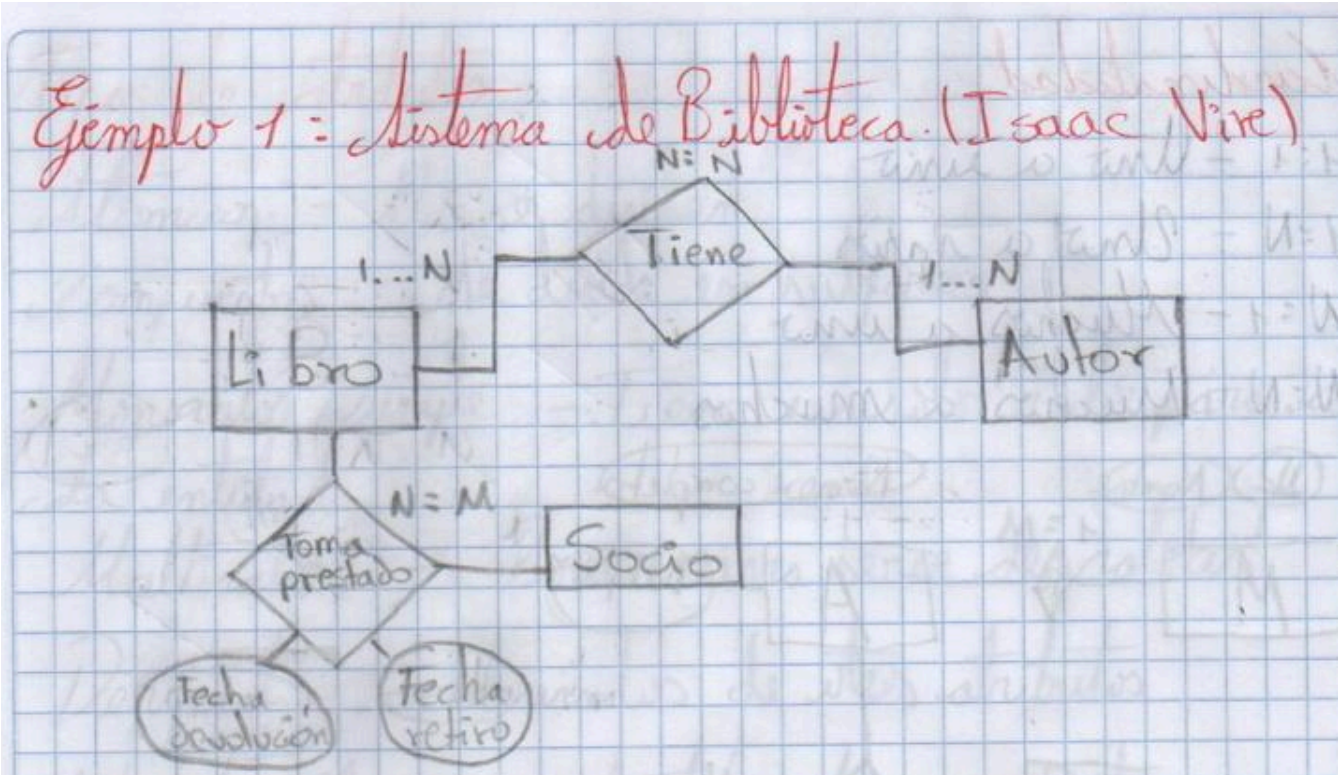
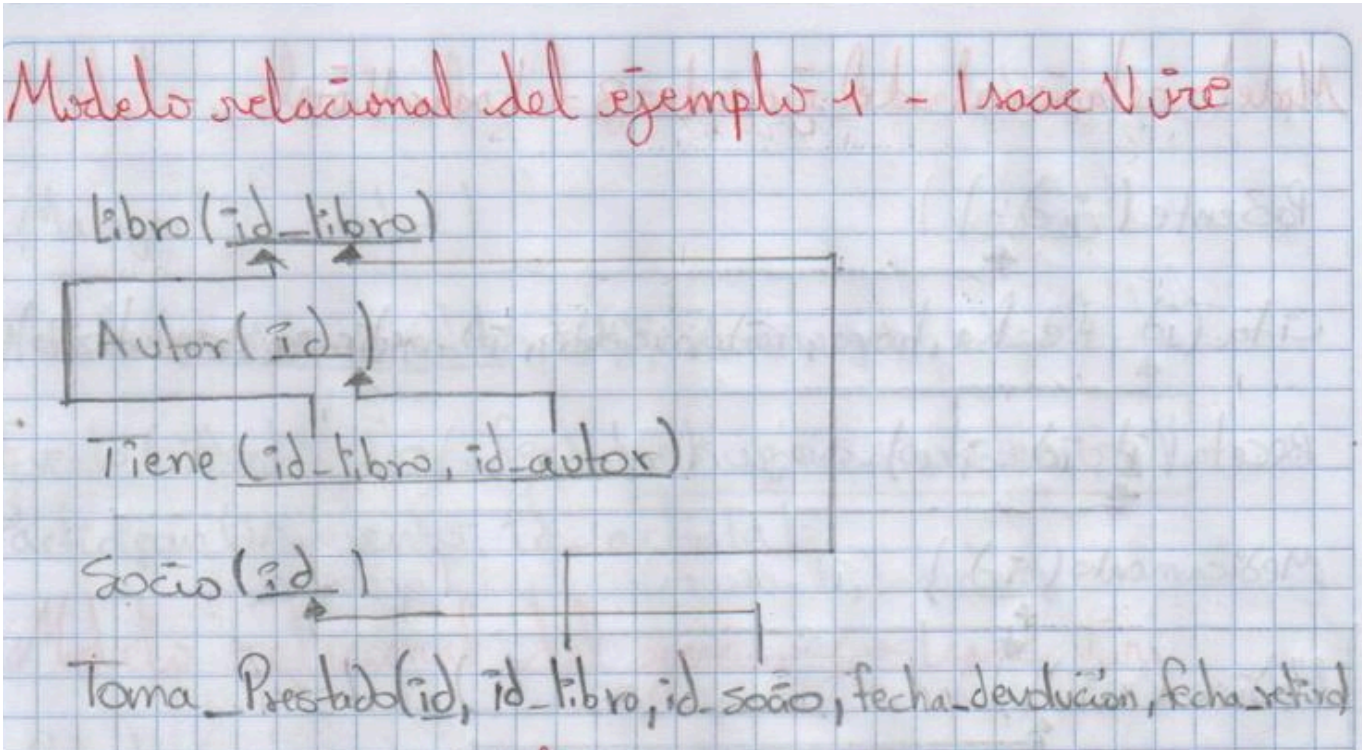
Aprendizaje Autónomo

Información General	
Facultad:	FEIRNNR
Carrera:	Computación
Semestre y paralelo:	3 Ciclo - Paralelo A
Periodo Académico:	Septiembre 2026-Febrero 2027
Estudiante:	Isaac Alejandro Vire Poma.
Enlace Ejercicios (GitHub/GitLab):	https://github.com/B3kk4r/Portafolio-de-Base-de-Datos.git
Materia:	Base de Datos
Docente Encargado:	René Guamán

Ejemplo 1: Sistema de Biblioteca

Una biblioteca registra libros y autores. Un libro puede tener varios autores y un autor varios libros. Los socios toman libros prestados; de cada préstamo interesan la fecha de retiro y la de devolución.





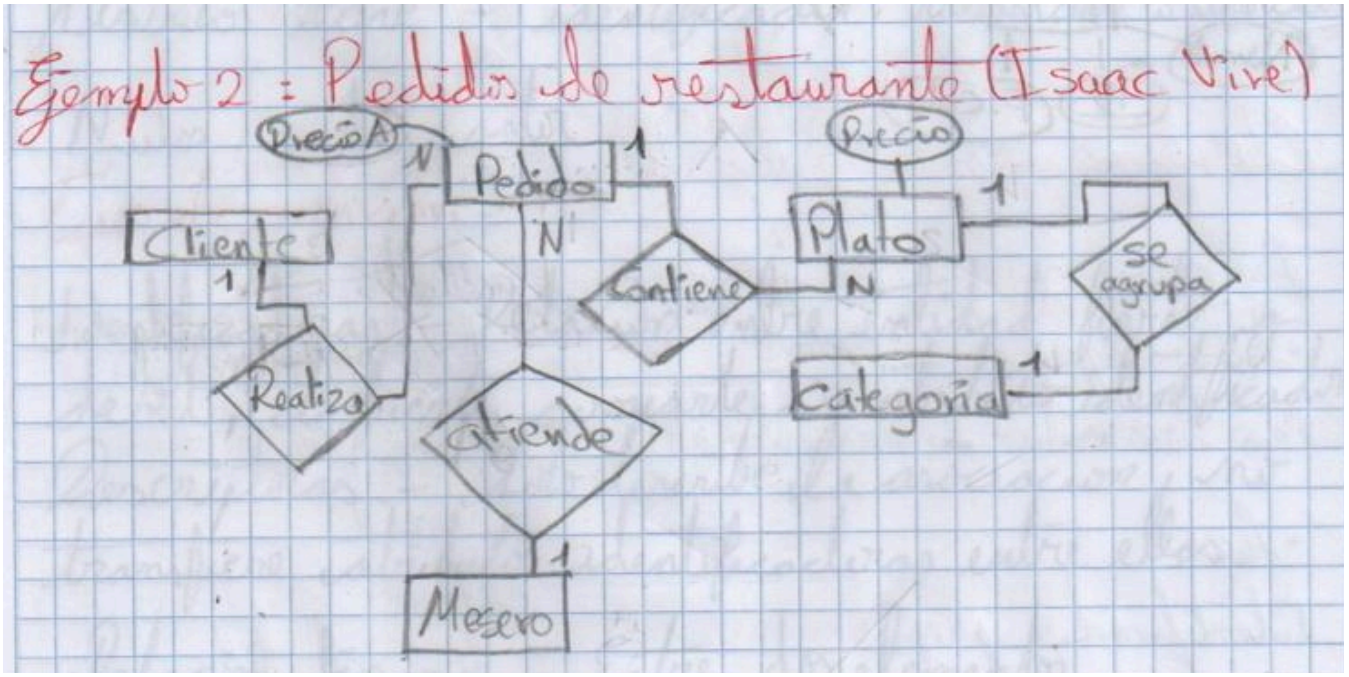
SQL

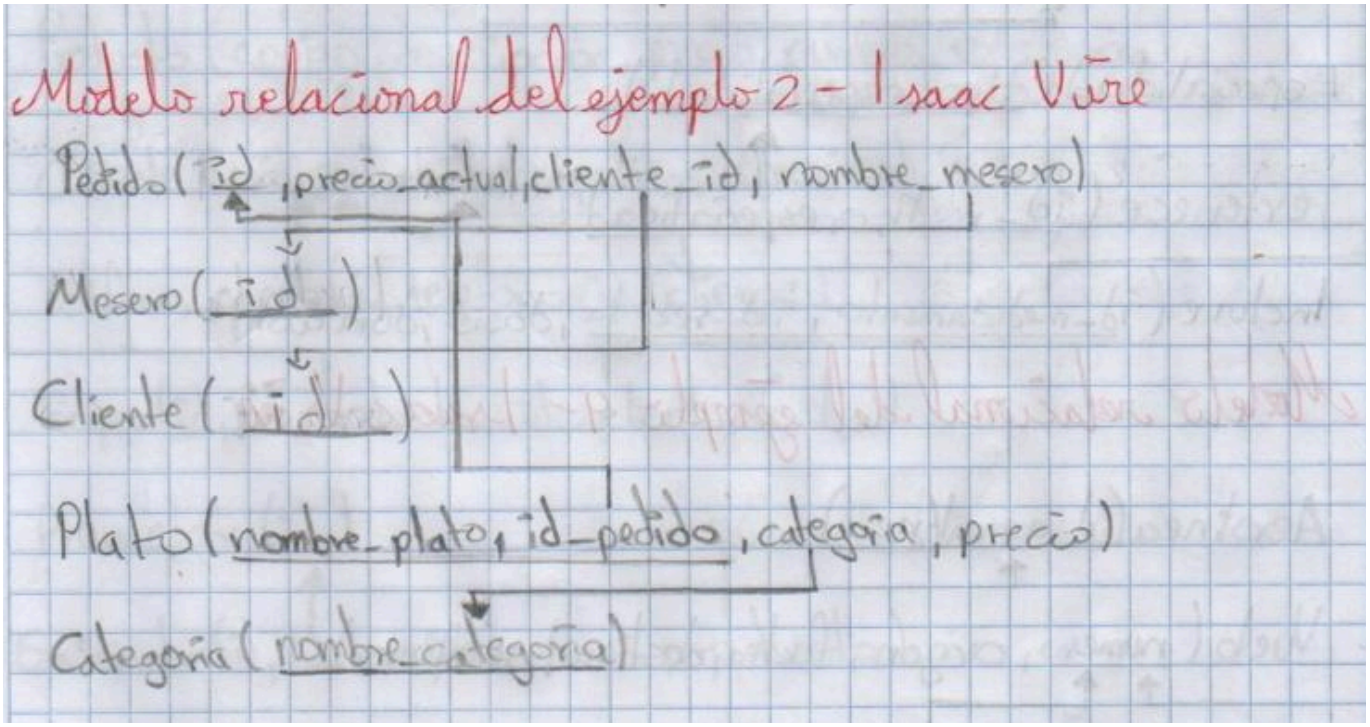
```
create table Libro(
	id_libro varchar (5) primary key
);
create table Autor(
	id varchar (5) primary key
);
create table Tiene(
	id_libro varchar (5) NOT NULL,
	id_autor varchar (5) NOT NULL,
	PRIMARY KEY (id_libro, id_autor),
	Foreign Key (id_libro) references Libro (id_libro),
	Foreign Key (id_autor) references Autor (id)
);
create table Socio (
	id varchar (5) primary key
);
create table Toma_Prestado(
	id varchar (5) primary key,
	fecha_devolucion DATE,
```

```
        fecha_retiro DATE,
        id_libro varchar (5) NOT NULL,
        id_socio varchar (5) NOT NULL,
        Foreign Key (id_libro) references Libro (id_libro),
        Foreign Key (id_socio) references Socio (id)
);
```

Ejemplo 2: Pedidos de Restaurante

Los clientes realizan pedidos que contienen varios platos con distinta cantidad. Los platos se agrupan en categorías. Cada pedido lo atiende un único mesero. El precio del plato debe registrarse tal como estaba al momento del pedido, aunque luego cambie.





SQL

```
create table Mesero(
	id varchar (15) primary key
);

create table Cliente(
	id varchar (15) primary key
);

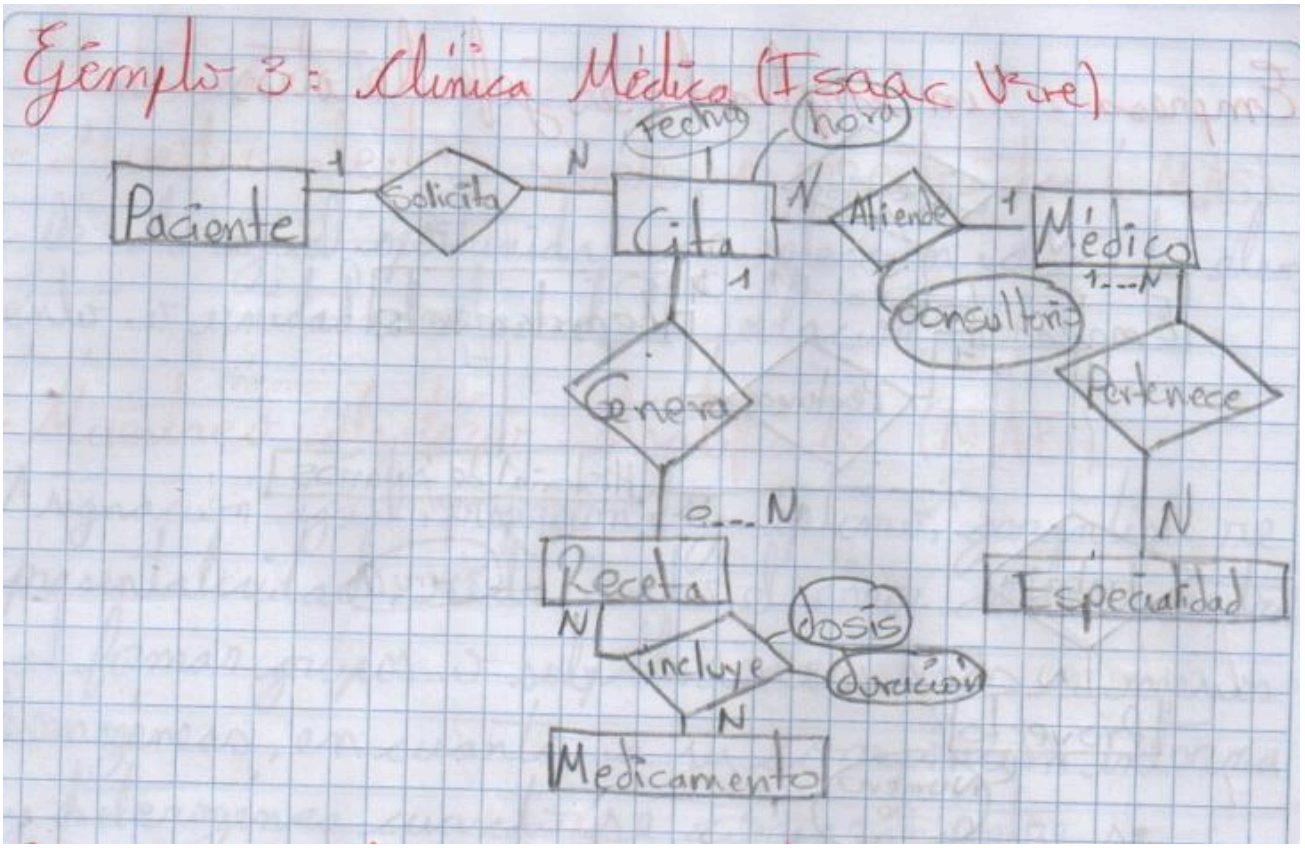
create table Categoria(
	nombre_categoria varchar (25) primary Key
);

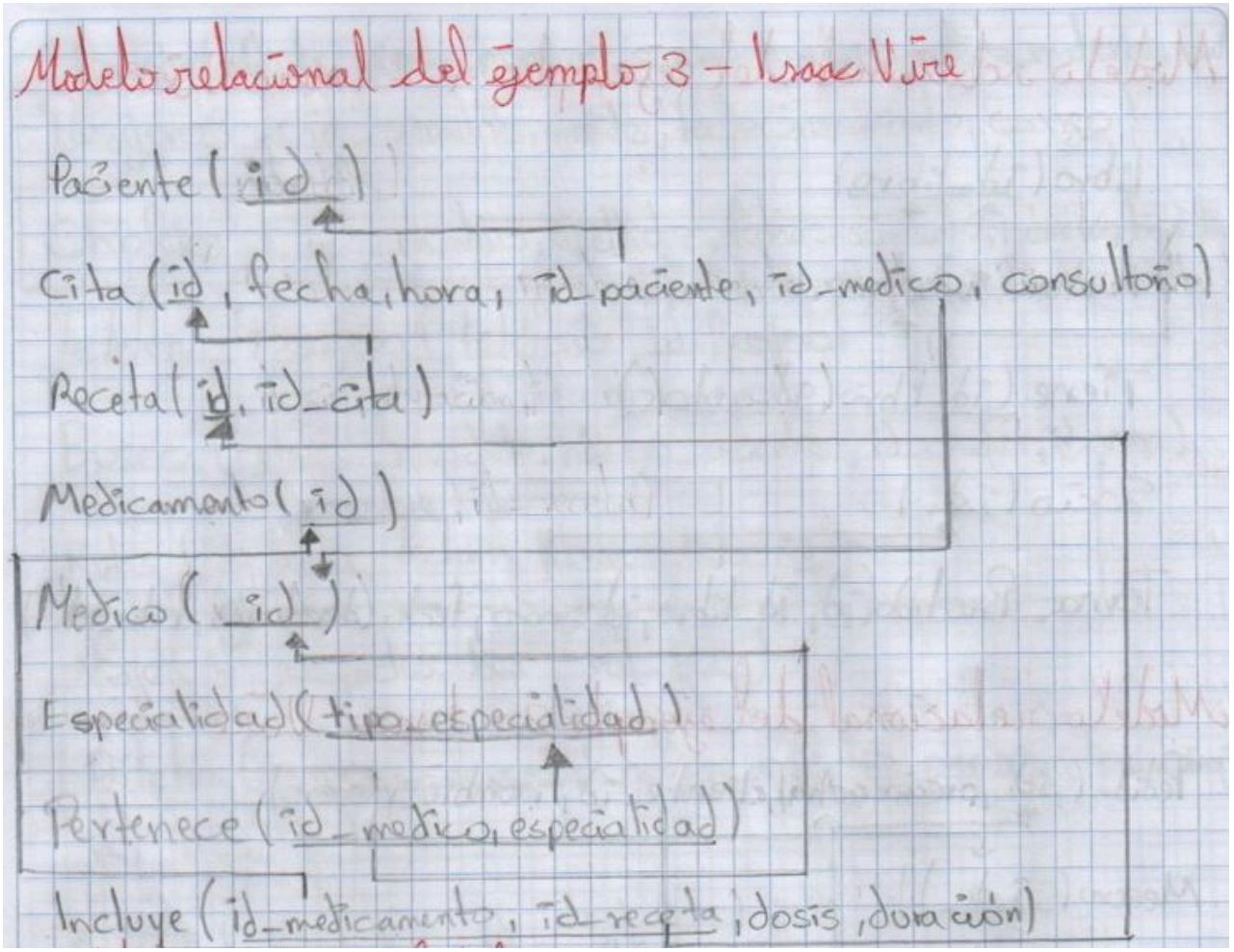
create table Pedido(
	id varchar (5) primary key,
	precio_actual double NOT NULL,
	cliente_id varchar (15) NOT NULL,
	nombre_mesero varchar (15) NOT NULL,
	Foreign Key (cliente_id) references Cliente (id),
	Foreign Key (nombre_mesero) references Mesero (id)
);
```

```
create table Plato(
	nombre_plato varchar (20) NOT NULL,
	id_pedido varchar (5) NOT NULL,
	categoria varchar (25) NOT NULL,
	precio int NOT NULL,
	PRIMARY KEY (nombre_plato, id_pedido),
	Foreign Key (id_pedido) references Pedido (id),
	Foreign Key (categoria) references Categoria (nombre_categoria)
);
```

Ejemplo 3: Clínica Médica

Los pacientes solicitan citas; cada cita la atiende un médico a un paciente en fecha, hora y consultorio. De algunas citas se genera una receta que incluye medicamentos con dosis y duración. Cada médico pertenece a una especialidad.





SQL

```
create table Paciente(
	id varchar (5) primary key
);

create table Medico(
	id varchar (5) primary key
);

create table Especialidad(
	tipo_especialidad varchar (20) primary key
);

create table Medicamento(
	id varchar (5) primary key
);
```

```
create table Cita(
	id varchar (5) primary key,
	fecha DATE NOT NULL,
	hora TIME NOT NULL,
	id_paciente varchar (5) NOT NULL,
	id_medico varchar (5) NOT NULL,
	consultorio varchar (5) NOT NULL,
	Foreign Key (id_paciente) references Paciente (id),
	Foreign Key (id_medico) references Medico (id)
);

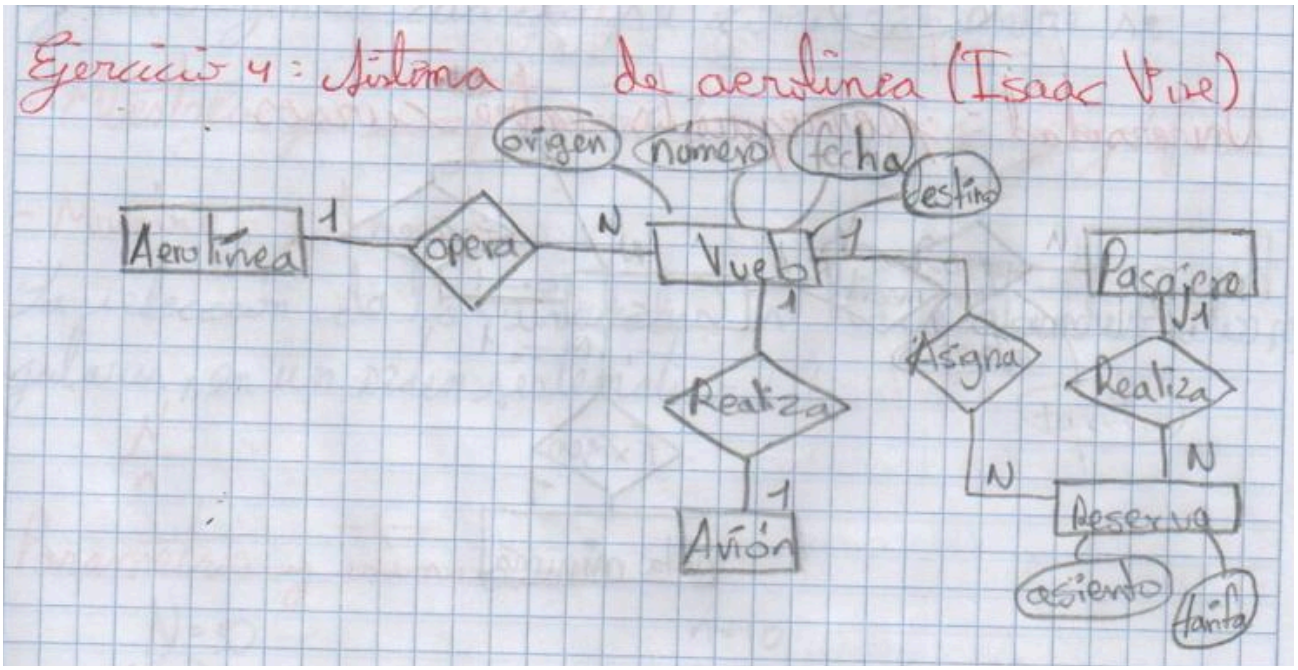
create table Pertenece(
	id_medico varchar (5) NOT NULL,
	especialidad varchar (20) NOT NULL,
	PRIMARY KEY (id_medico, especialidad),
	Foreign Key (id_medico) references Medico (id),
	Foreign Key (especialidad) references Especialidad (tipo_especialidad)
);

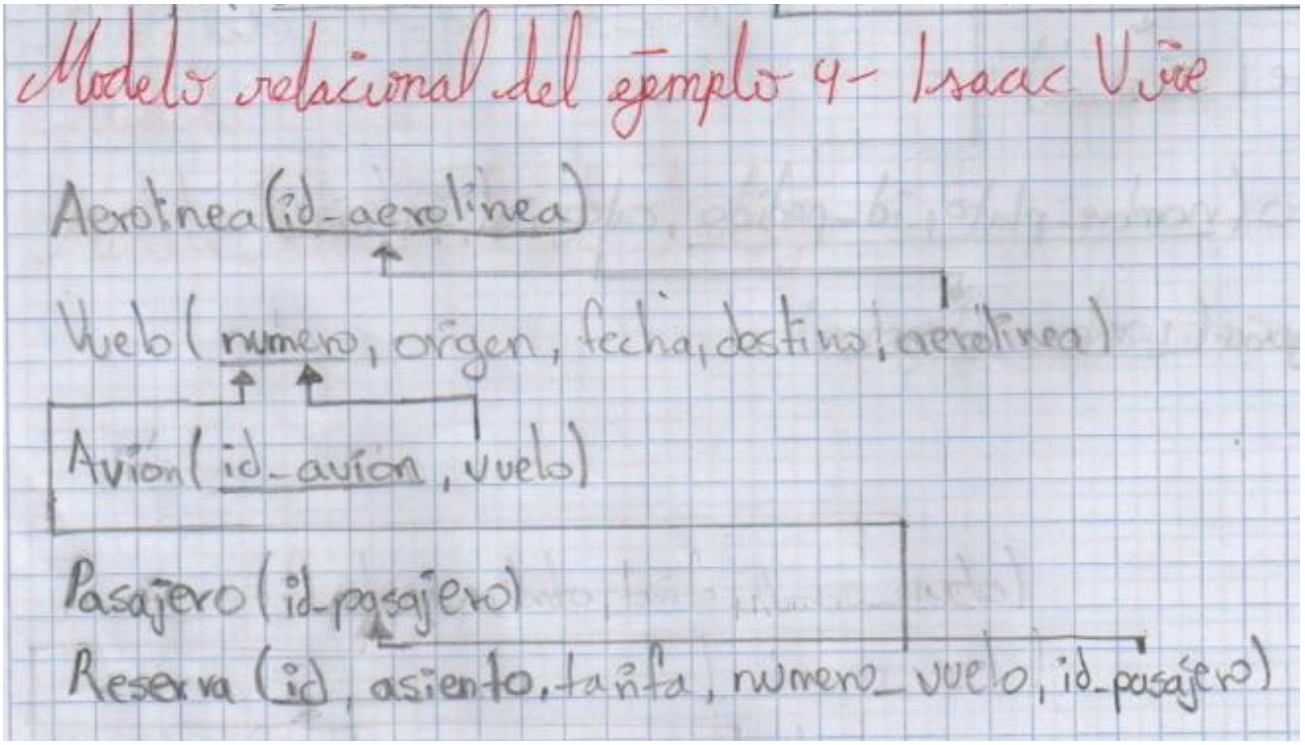
create table Receta(
	id varchar (5) primary key,
	id_cita varchar (5) NOT NULL,
	Foreign Key (id_cita) references Cita (id)
);

create table Incluye(
	id_medicamento varchar (5) NOT NULL,
	id_receta varchar (5) NOT NULL,
	dosis varchar (20) NOT NULL,
	duracion varchar (20) NOT NULL,
	PRIMARY KEY (id_medicamento, id_receta),
	Foreign Key (id_medicamento) references Medicamento (id),
	Foreign Key (id_receta) references Receta (id)
);
```

Ejemplo 4: Sistema de Aerolínea

Una aerolínea opera vuelos (cada uno con número, fecha, origen y destino), cada vuelo lo realiza un avión. Los pasajeros realizan reservas para un vuelo específico, indicando asiento y tarifa pagada en el momento de la reserva.





SQL

```
create table Aerolinea(
	id_aerolinea varchar (10) primary key
);
```

```
create table Vuelo(
	numero varchar (10) primary key,
	origen varchar (20) NOT NULL,
	fecha DATE NOT NULL,
	destino varchar (20) NOT NULL,
	aerolinea varchar (10) NOT NULL,
	Foreign Key (aerolinea) references Aerolinea (id_aerolinea)
);

create table Avion(
	id_avion varchar (20) primary key,
	vuelo varchar (10) NOT NULL,
	Foreign Key (vuelo) references Vuelo (numero)
);

create table Pasajero(
	id_pasajero varchar (20) primary key
);

create table Reserva(
	id varchar (10) primary key,
	asiento int NOT NULL,
	tarifa double NOT NULL,
	numero_vuelo varchar (10) NOT NULL,
	id_pasajero varchar (20) NOT NULL,
	Foreign Key (numero_vuelo) references Vuelo (numero),
	Foreign Key (id_pasajero) references Pasajero (id_pasajero)
);
```

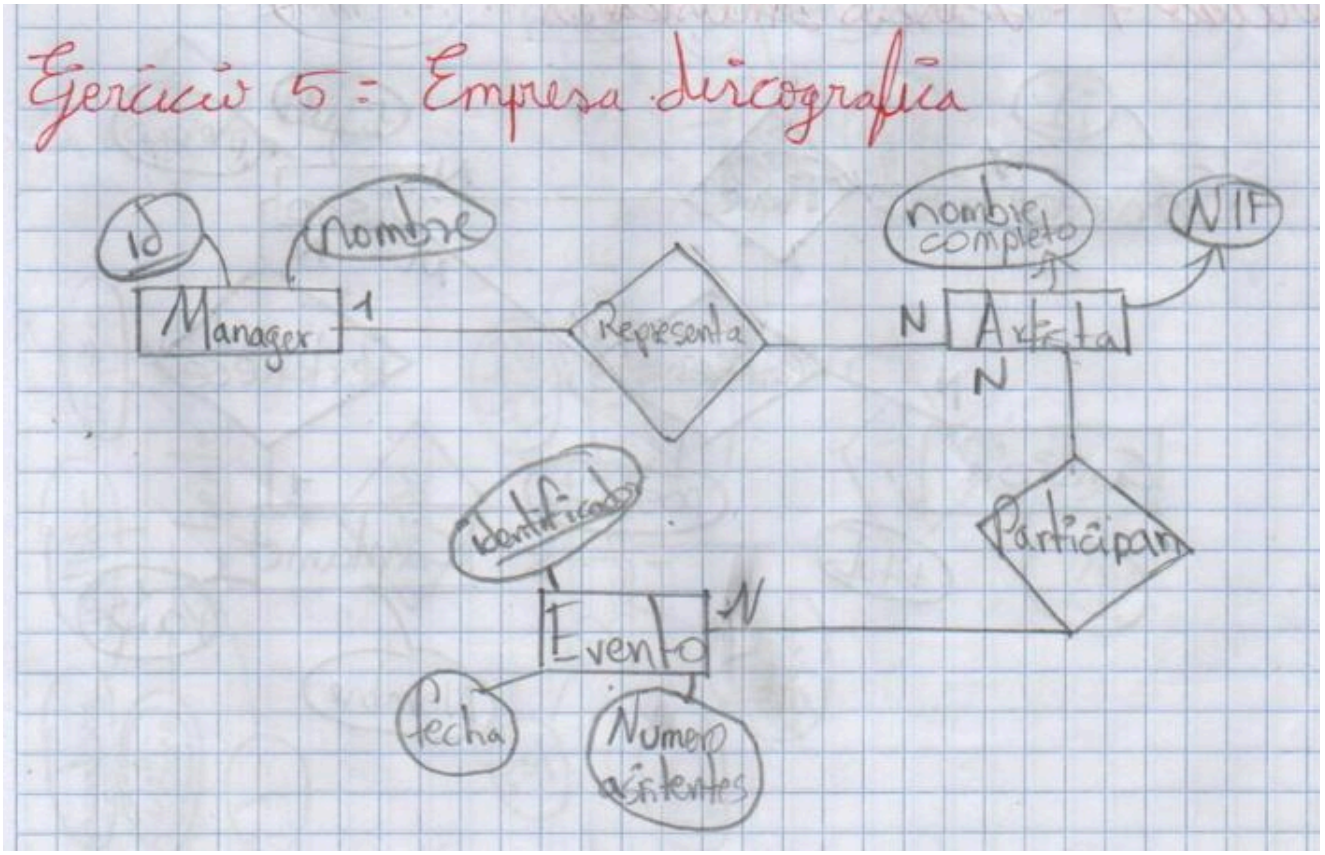
Ejercicios Propuestos

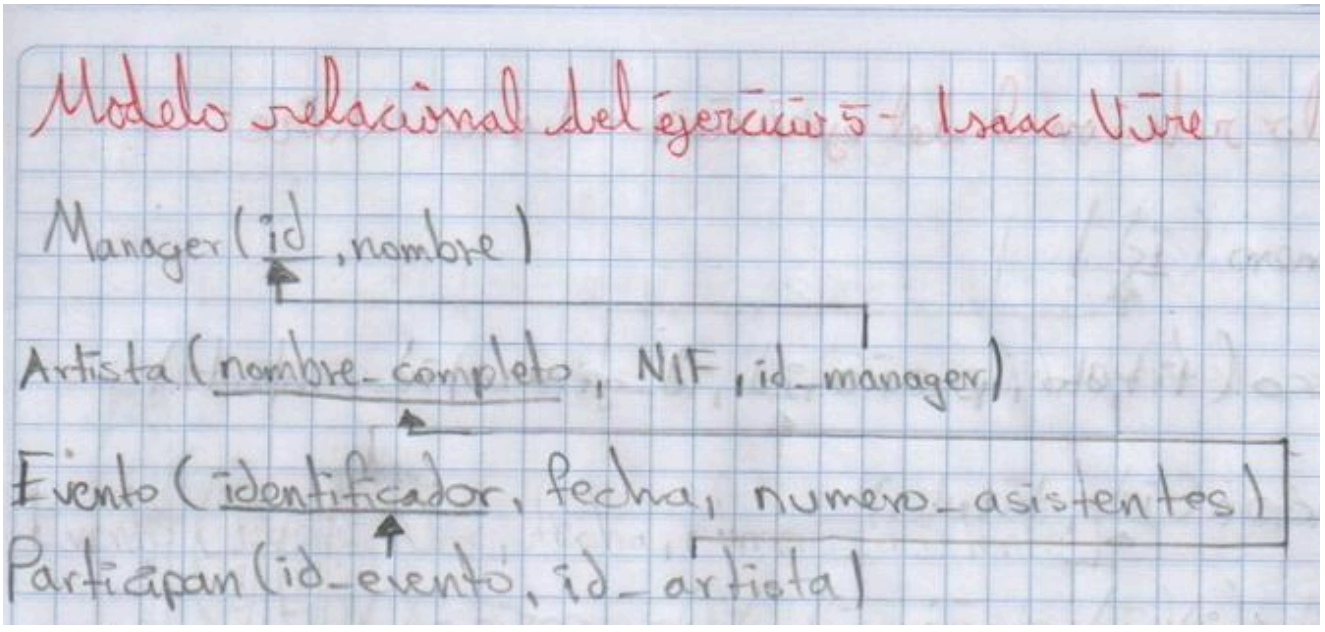
Ejercicio 5: Empresa Discográfica

Una empresa discográfica necesita modelar los datos sobre sus diferentes recursos según las siguientes características:

- Manager: de cada mánager se almacena un identificador de manager y su nombre. Un manager representa a una serie de artistas.

- Artista: un artista es representado por un manager. De los artistas se almacena su nombre completo (usando un único atributo) y su NIF.
- Evento de promoción: los artistas participan en eventos de promoción para dar a conocer sus trabajos. En un evento de promoción pueden participar varios artistas. De un evento se almacena un identificador único, la fecha de celebración y el número de asistentes.





SQL

```
create table Manager (
```

```
	id varchar (5) primary key,
	nombre varchar (20) NOT NULL
);

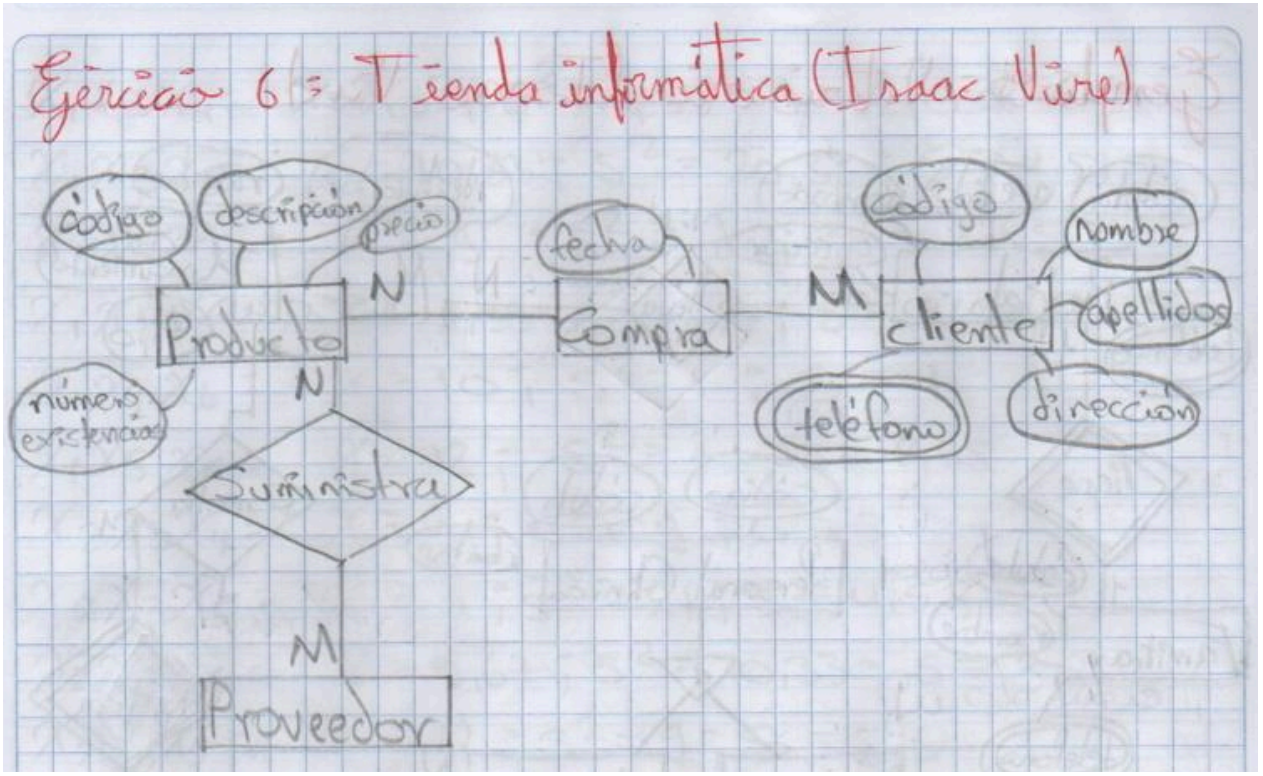
create table Artista (
	nombre_completo varchar(50) primary key,
	NIF varchar (10) NOT NULL,
	id_manager varchar (5) NOT NULL,
	Foreign Key (id_manager) references Manager (id)
);

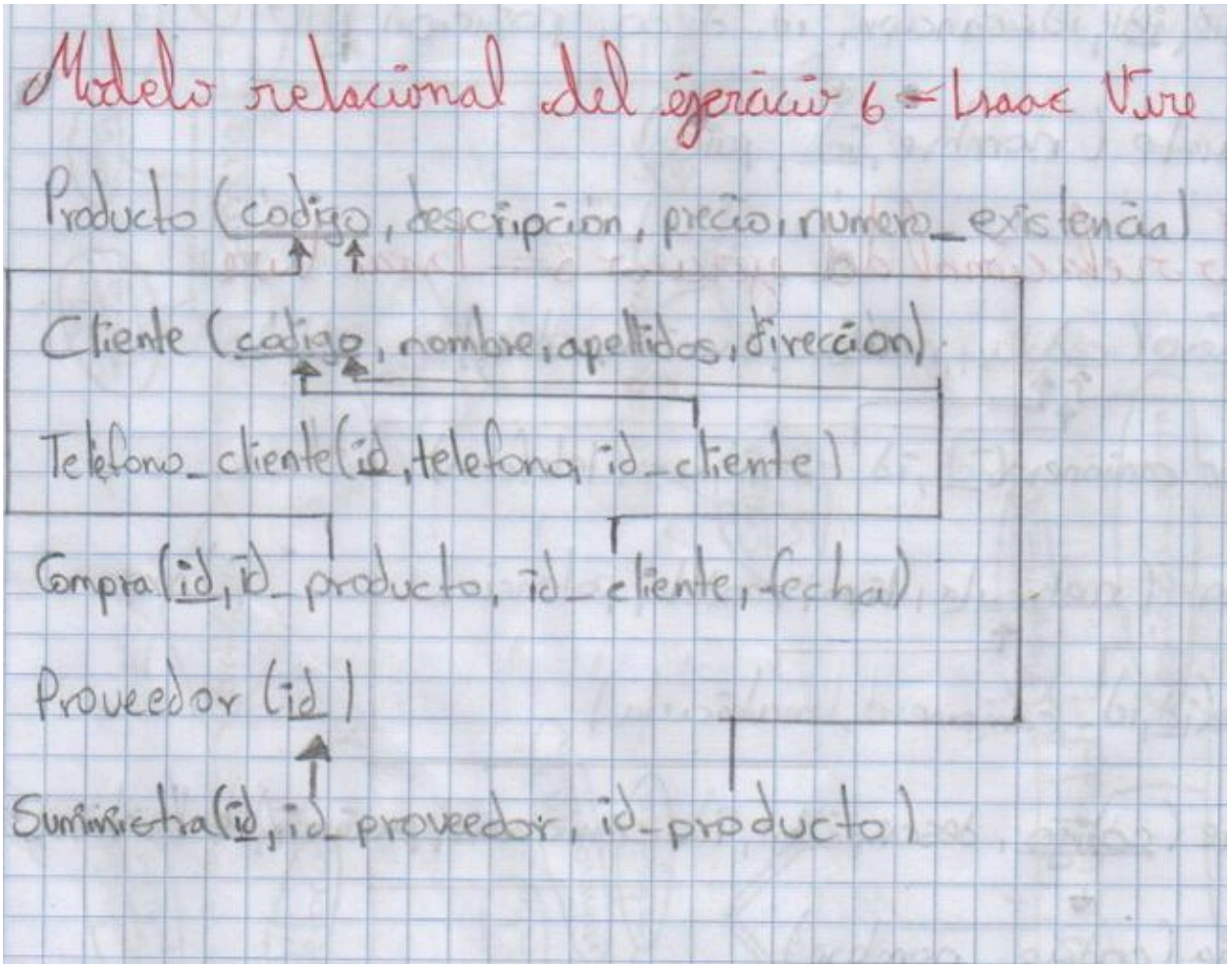
create table Evento (
	identificador varchar (10) primary key,
	fecha DATE NOT NULL,
	numero_asistentes int NOT NULL
);

create table Participan (
	id_evento varchar (10) NOT NULL,
	id_artista varchar (50) NOT NULL,
	PRIMARY KEY(id_evento, id_artista),
	Foreign Key (id_evento) references Evento (identificador),
	Foreign Key (id_artista) references Artista (nombre_completo)
);
```

Ejercicio 6: Tienda Informática

Se desea crear una aplicación para la gestión de una tienda informática. La tienda dispone de una serie de productos que se venden a los clientes. De cada producto informático se desea guardar el código, descripción, precio y número de existencias. De cada cliente se desea guardar el código, nombre, apellidos, dirección y número telefónico. Un cliente puede comprar varios productos en la tienda y un mismo producto puede ser comprado por varios clientes. Cada vez que se compre un artículo quedará registrada la compra en la base de datos junto con la fecha en la que se compró el artículo. La tienda tiene contactos con varios proveedores que son los que suministran los productos. Un mismo producto puede ser suministrado por varios proveedores. De cada proveedor se desea guardar el código, nombres, apellidos, dirección, provincia y número de teléfono.


Ejercicio 6 : Tienda informática (Isaac Vivo)
código
descripción
precio
Producto
N
fecha
Compra
M
código
nombre
cliente
apellidos
teléfono
dirección
número existencias
N
Suministra
M
Proveedor


Modelo relacional del ejercicio 6 - Isaac Vivo
Producto (codigo, descripcion, precio, numero_existencias)
Cliente (codigo, nombre, apellidos, direccion)
Telefono_cliente(id, telefono, id_cliente)
Compra(id, id_producto, id_cliente, fecha)
Proveedor (id)
Suministra(id, id_proveedor, id_producto)

SQL

```
create table Producto (
	codigo varchar (10) primary key,
	descripcion varchar (50) NOT NULL,
	precio int NOT NULL,
	numero_existencia int NOT NULL
);

create table Cliente (
	codigo varchar (10) primary key,
	nombre varchar (15) NOT NULL,
	apellidos varchar (30) NOT NULL,
	direccion varchar (15) NOT NULL
);

create table Telefono_Cliente (
	id varchar (5) primary key,
	telefono varchar (10),
	id_cliente varchar (10) NOT NULL,
	Foreign Key (id_cliente) references Cliente (codigo)
);

create table Compra (
	id varchar (10) primary key,
	id_producto varchar (10) NOT NULL,
	id_cliente varchar (10) NOT NULL,
	fecha DATE NOT NULL,
	Foreign Key (id_cliente) references Cliente (codigo),
	Foreign Key (id_producto) references Producto (codigo)
);

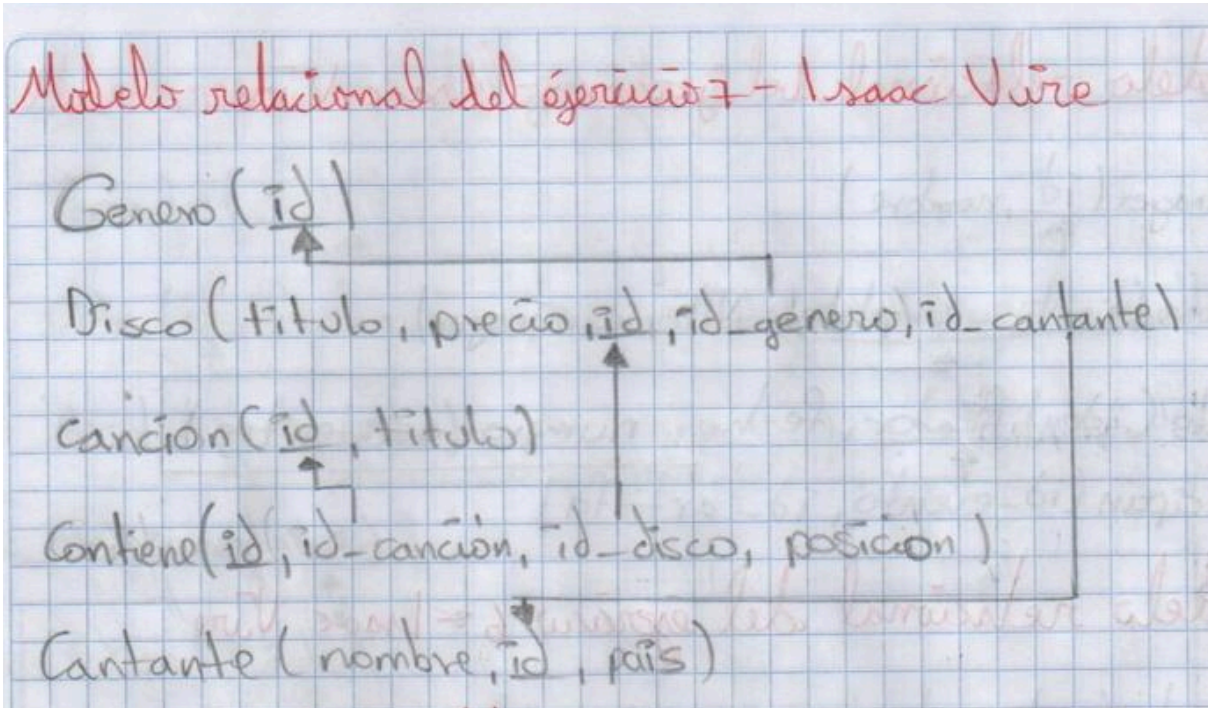
create table Proveedor (
	id varchar (10) primary key
);

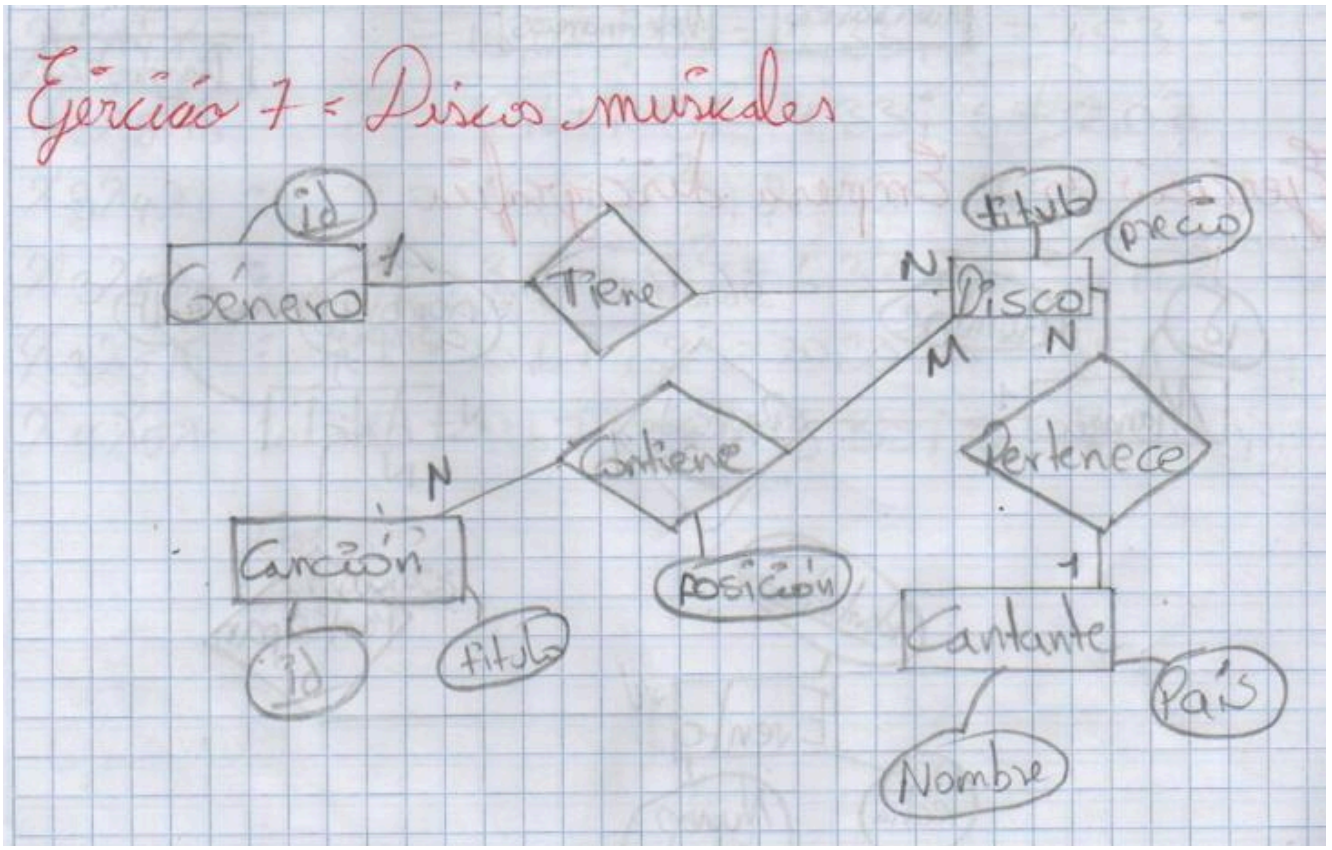
create table Suministra (
```

```
        id varchar (10) primary key,
        id_proveedor varchar (10) NOT NULL,
        id_producto varchar (10) NOT NULL,
        Foreign Key (id_proveedor) references Proveedor (id),
        Foreign Key (id_producto) references Producto (codigo)
);
```

Ejercicio 7: Discos Musicales

- La tienda vende discos de diferentes géneros musicales y cantantes
- Cada género musical tiene un identificador
- Cada disco tiene titulo, genero musical, precio y cantante
- De un cantante se registra el nombre y su país, Todo disco pertenece a un cantante y un cantante puede tener muchos discos. Además un disco tiene un conjunto de canciones(identificador y título)
- Un disco tiene muchas canciones y una canción puede estar en varios discos. Interesa conocer la posición de una canción en un determinado disco





SQL

```
create table Genero (

	id varchar (10) primary key

);
```

```
create table Cantante (
	id varchar (10) primary key,
	nombre varchar (15) NOT NULL,
	pais varchar (25) NOT NULL
);

create table Disco (
	id varchar (10) primary key,
	titulo varchar (20) NOT NULL,
	precio int NOT NULL,
	id_genero varchar (10) NOT NULL,
	id_cantante varchar (10) NOT NULL,
	Foreign Key (id_genero) references Genero (id),
	Foreign Key (id_cantante) references Cantante (id)
);

create table Cancion (
	id varchar (10) primary key,
	titulo varchar (20) NOT NULL
);

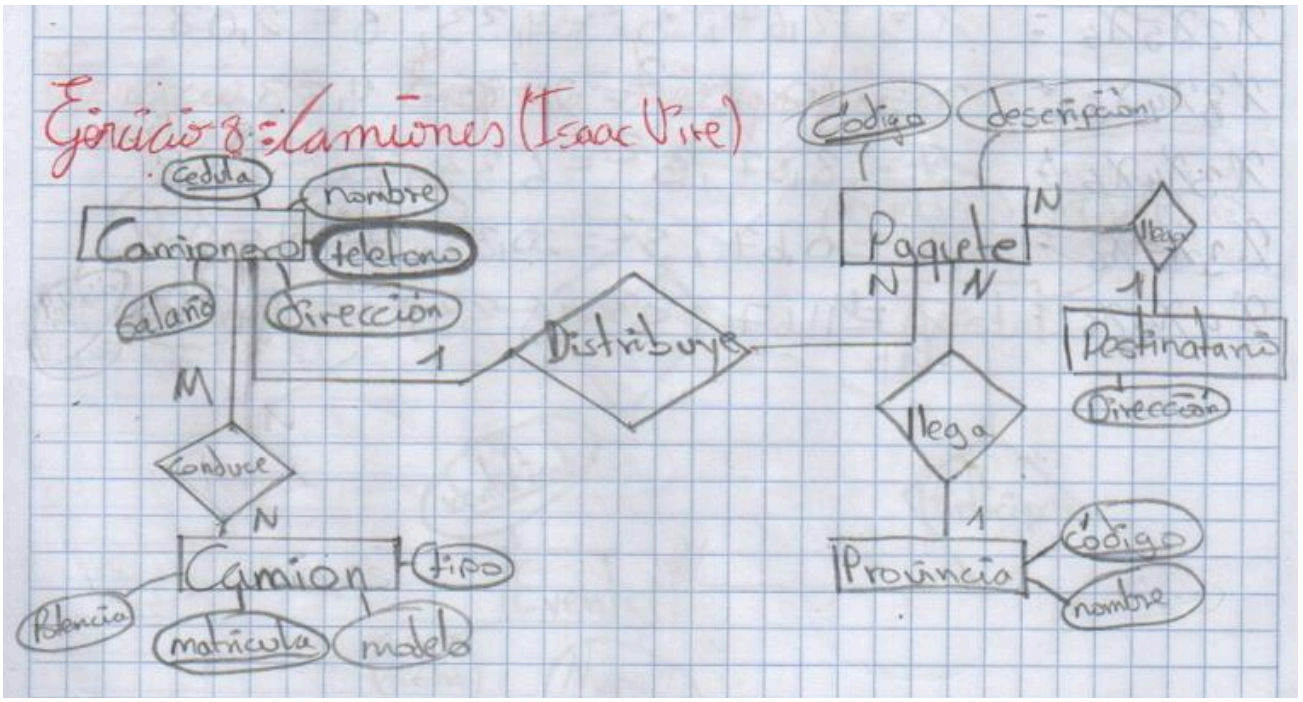
create table Contiene (
	id varchar (10) primary key,
	id_cancion varchar (10) NOT NULL,
	id_disco varchar (10) NOT NULL,
	posicion int NOT NULL,
	Foreign Key (id_cancion) references Cancion (id),
	Foreign Key (id_disco) references Disco (id)
);
```

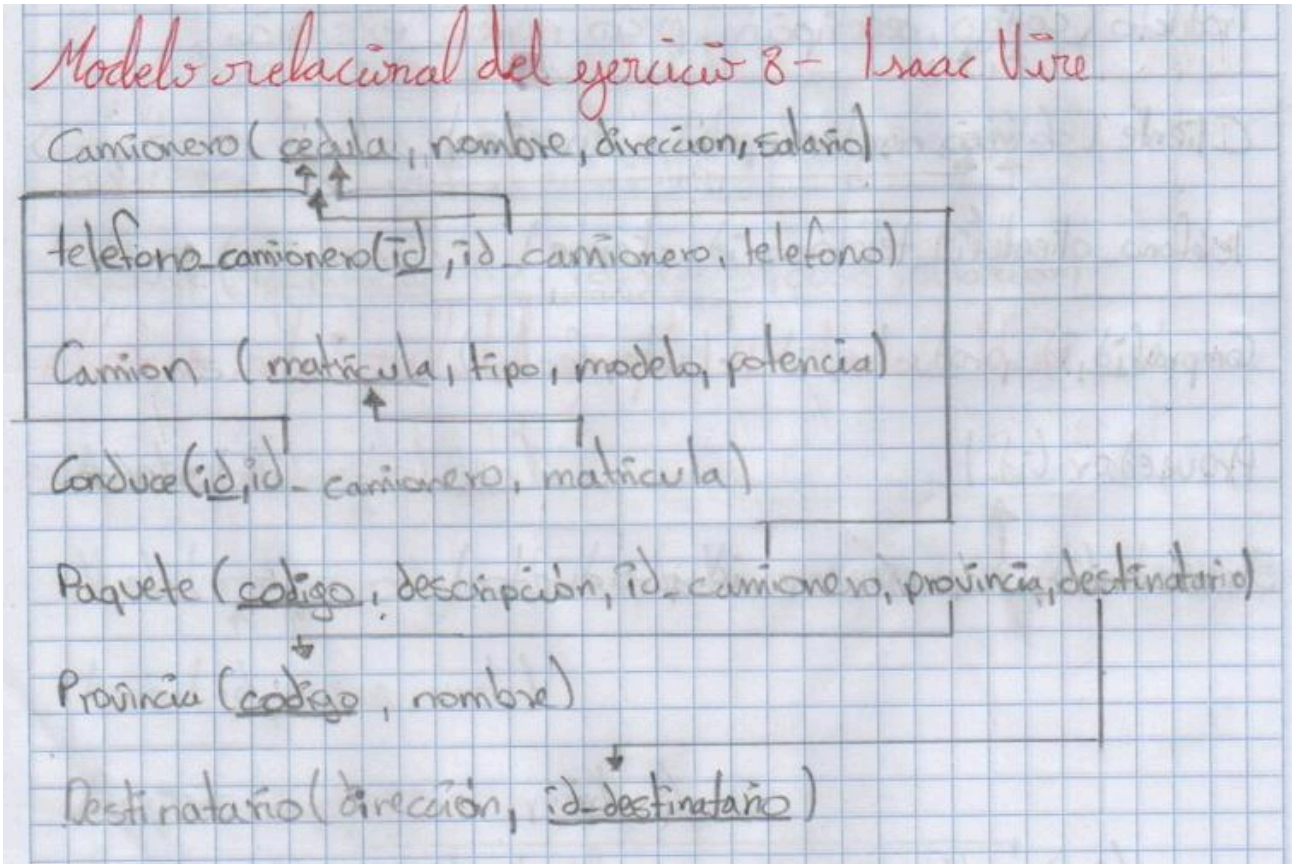
Ejercicio 8: Camiones

Se desea informatizar la gestión de una empresa de transporte que reparte paquetes por toda España. Los encargados de llevar los paquetes son los camioneros, de los que se quiere guardar la cédula, nombre, teléfono, dirección, salario y población en que vive. De los paquetes transportados interesa conocer el código del paquete, descripción, destinatario y dirección del destinatario.

Un camionero distribuye muchos paquetes, y un paquete sólo puede ser distribuido por un camionero. De las provincias a las que llegan los paquetes interesa guardar el código de provincia y el nombre. Un paquete sólo puede llegar a una provincia. Sin embargo, a una provincia pueden llegar varios paquetes.

De los camiones que llevan los camioneros, interesa conocer la matrícula, modelo, tipo y potencia. Un camionero puede conducir diferentes camiones en fechas diferentes, y un camión puede ser conducido por varios camioneros.





SQL

```
create table Camionero (
	cedula varchar (10) primary key,
	nombre varchar (20) NOT NULL,
	direccion varchar (50) NOT NULL,
	salario double NOT NULL
);
create table Telefono_camionero (
	id varchar (10) primary key,
	id_camionero varchar (10) NOT NULL,
	telefono varchar (10),
	Foreign Key (id_camionero) references Camionero (cedula)
);
create table Camion (
	matricula varchar (10) primary key,
	tipo varchar (10) NOT NULL,
	modelo varchar (10) NOT NULL,
	potencia int NOT NULL
);
create table Conduce (
	id varchar (10) primary key,
	id_camionero varchar (10) NOT NULL,
	matricula varchar (10) NOT NULL,
	Foreign Key (id_camionero) references Camionero (cedula),
	Foreign Key (matricula) references Camion (matricula)
);

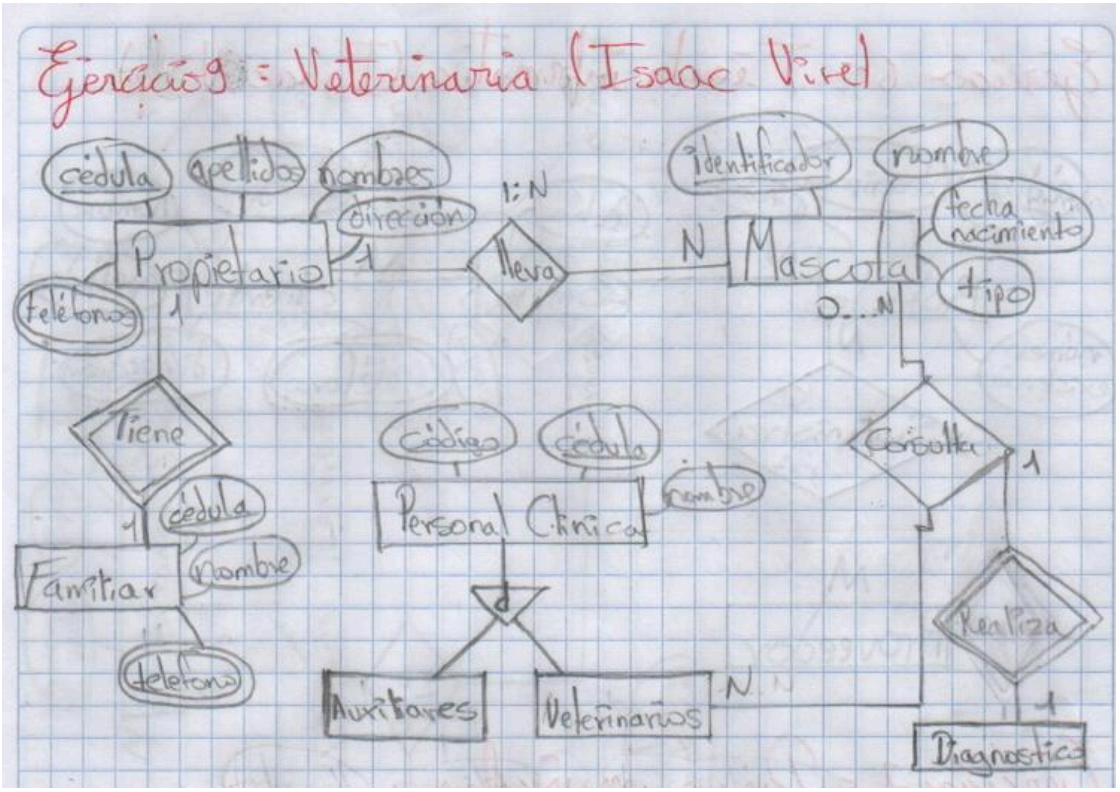
create table Provincia (
	codigo varchar (20) primary key,
	nombre varchar (30) NOT NULL
);
create table Destinatario (
	id_destinatario varchar (10) primary key,
	direccion varchar (50) NOT NULL
);
```

```
create table Paquete (
	codigo varchar (10) primary key,
	descripcion varchar (50) NOT NULL,
	id_camionero varchar (10) NOT NULL,
	provincia varchar (20) NOT NULL,
	destinatario varchar (10) NOT NULL,
	Foreign Key (id_camionero) references Camionero (cedula),
	Foreign Key (provincia) references Provincia (codigo),
	Foreign Key (destinatario) references Destinatario (id_destinatario)
);
```

Ejercicio 9: Veterinaria

- Del propietario interesa: cédula, apellidos, nombres, dirección , teléfonos
- De la mascota interesa: identificador, nombre, fecha de nacimiento, tipo
- Un propietario puede llevar una o varias mascotas
- Una mascota la lleva uno y solo un propietario
- Del personal de la clínica veterinaria interesa un código, el cédula, nombre
- Entre el personal de la clínica se distingue a los veterinarios y los auxiliares, pudiendo haber otros tipos
    - Del veterinario se guarda además su fecha_alta y especialidad
    - De los auxiliares interesa su base de cotización
- Las mascotas pasan consulta con los veterinarios. De manera que:
    - un veterinario puede pasar consulta a ninguna o a varias mascotas, y
    - una misma mascota de la clínica ha pasado consulta una o más veces en fechas diferentes con varios veterinarios.
    - Al pasar consulta se realiza un diagnóstico.
- Cada propietario tiene un familiar de contacto para casos de emergencia.
    - De este familiar interesa la cédula, nombre y teléfono.
    - Si el propietario se da de alta en la clínica veterinaria, el familiar ya no interesa.
    - El familiar es contacto de un solo propietario

Ejercicio 9 : Veterinaria (Isaac Vire)



Modelo relacional del ejercicio 9 - Isaac Vire

Propietario(cedula, apellido, nombre, direccion, id_familiar)

Telefono_propietario(id, id_propietario, telefono)

Mascota(identificador, nombre, fecha_nacimiento, tipo, id_propietario)

Familiar(cedula, nombre)

Telefono_familiar(id, cedula, telefono_familiar)

Personal_clinica(codigo, cedula, nombre)

Veterinario(id_veterinario)

Auxiliar(id_auxiliar)

Consulta(id, id_veterinario, id_mascota, diagnostico)

Diagnostico(id)

SQL

```
create table Familiar (
	cedula varchar (20) primary key,
	nombre varchar (20) NOT NULL
);

create table Propietario (
	cedula varchar (10) primary key,
	apellido varchar (20) NOT NULL,
	nombre varchar (20) NOT NULL,
	direccion varchar (30) NOT NULL,
	id_familiar varchar (20) NOT NULL,
	Foreign Key (id_familiar) references Familiar (cedula)
);

create table Telefono_propietario (
	id varchar (10) primary key,
	id_propietario varchar (10) NOT NULL,
	telefono varchar (10) NOT NULL,
	Foreign Key (id_propietario) references Propietario (cedula)
);

create table Mascota (
	identificador varchar (15) primary key,
	nombre varchar (20) NOT NULL,
	fecha_nacimiento DATE NOT NULL,
	tipo varchar (20) NOT NULL,
	id_propietario varchar (10) NOT NULL,
	Foreign Key (id_propietario) references Propietario (cedula)
);

create table Telefono_familiar (
	id varchar (10) primary key,
	cedula varchar (20) NOT NULL,
	telefono_familiar varchar (20) NOT NULL,
	Foreign Key (cedula) references Familiar (cedula)
```

```
);

create table Personal_clinica (
	codigo varchar (10) primary key,
	cedula varchar (10) NOT NULL,
	nombre varchar (20) NOT NULL
);

create table Veterinario (
	id_veterinario varchar (10) primary key,
	Foreign Key (id_veterinario) references Personal_clinica (codigo)
);

create table Auxiliar (
	id_auxiliar varchar (10) primary key,
	Foreign Key (id_auxiliar) references Personal_clinica (codigo)
);

create table Diagnostico (
	id varchar (50) primary key
);

create table Consulta (
	id varchar (10) primary key,
	id_veterinario varchar (10) NOT NULL,
	id_mascota varchar (15) NOT NULL,
	diagnostico varchar (50) NOT NULL,
	Foreign Key (id_veterinario) references Personal_clinica (codigo),
	Foreign Key (id_mascota) references Mascota (identificador),
	Foreign Key (diagnostico) references Diagnostico (id)
);
```